

Aufgabe Woche 6 (Lösung)

Lösung: Shiny-App für S&P 500 Portfolio Entscheid

1. Benötigte Pakete installieren & laden

```
install.packages(c("shiny", "quantmod", "TTR", "dygraphs", "dplyr",  
"shinyWidgets"))  
library(shiny)  
library(quantmod) # Für Yahoo Finance-Daten  
library(TTR)      # Für RSI und Moving Averages  
library(dygraphs) # Interaktive Plots  
library(dplyr)    # Datenverarbeitung
```

2. Shiny-App-Code (*app.R*)

```
# UI-Definition  
ui <- fluidPage(  
  titlePanel("ETF Kauf-/Verkaufsempfehlung (S&P 500)"),  
  sidebarLayout(  
    sidebarPanel(  
      selectInput("etf", "Wähle ETF:", choices = c("SPY", "VOO"), selected =  
"SPY"),  
      dateRangeInput("dates", "Zeitraum:", start = Sys.Date() - 365*5, end =  
Sys.Date()),  
      sliderInput("rsi_period", "RSI-Periode:", min = 5, max = 30, value =  
14),  
      actionButton("update", "Aktualisieren")  
    ),  
    mainPanel(  
      dygraphOutput("price_plot"),  
      dygraphOutput("rsi_plot"),  
      h3("Empfehlung:"),  
      verbatimTextOutput("decision")  
    )  
  )  
)  
  
# Server-Logik  
server <- function(input, output) {  
  data <- eventReactive(input$update, {  
    getSymbols(input$etf, src = "yahoo", from = input$dates[1], to =  
input$dates[2], auto.assign = FALSE)  
  })  
  
  # Technische Indikatoren berechnen  
  indicators <- reactive({  
    df <- data()
```

```

df$MA_50 <- SMA(Cl(df), n = 50)
df$MA_200 <- SMA(Cl(df), n = 200)
df$RSI <- RSI(Cl(df), n = input$rsi_period)
df
})

# Plot: Kurs + Moving Averages
output$price_plot <- renderDygraph({
  df <- indicators()
  dygraph(cbind(Cl(df), df$MA_50, df$MA_200), main = "Kurs mit Moving
Averages") %>%
  dySeries("Close", label = "Kurs") %>%
  dySeries("MA_50", label = "50-Tage-MA") %>%
  dySeries("MA_200", label = "200-Tage-MA") %>%
  dyRangeSelector()
})

# Plot: RSI
output$rsi_plot <- renderDygraph({
  df <- indicators()
  dygraph(df$RSI, main = "RSI") %>%
  dyLimit(70, "Überkauft", color = "red") %>%
  dyLimit(30, "Überverkauft", color = "green") %>%
  dyRangeSelector()
})

# Entscheidungslogik
output$decision <- renderText({
  df <- tail(indicators(), 1)
  ma_signal <- ifelse(df$MA_50 > df$MA_200, "Golden Cross (Kaufen)", "Death
Cross (Verkaufen)")
  rsi_signal <- case_when(
    df$RSI > 70 ~ "RSI > 70 (Überkauft → Verkaufen)",
    df$RSI < 30 ~ "RSI < 30 (Überverkauft → Kaufen)",
    TRUE ~ "RSI neutral (Halten)"
  )
  paste("Moving Averages:", ma_signal, "\nRSI:", rsi_signal)
})
}

# App starten
shinyApp(ui, server)

```

Erklärung der Logik

1. Datenabfrage

- Die App lädt historische Daten von Yahoo Finance (`getSymbols`) für den ausgewählten ETF (`SPY` oder `VOO`).

2. Technische Indikatoren

- **Moving Averages (MA):**
 - Golden Cross (50-Tage-MA > 200-Tage-MA) → Kaufsignal.
 - Death Cross (50-Tage-MA < 200-Tage-MA) → Verkaufsignal.
- **RSI:**
 - RSI > 70 → Überkauft (Verkaufen).
 - RSI < 30 → Überverkauft (Kaufen).

3. Visualisierung

- **dygraphs** zeigt interaktive Plots für Kursverlauf + MAs und RSI mit Schwellenwerten.

4. Empfehlung

- Die App kombiniert MA- und RSI-Signale für eine finale Entscheidung.

Wie testen?

1. Kopiere den Code in eine Datei `app.R`.
2. Führe sie in RStudio aus (`Run App`).
3. Wähle einen ETF und Zeitraum, dann klicke auf *Aktualisieren*.

Beispiel-Ausgabe

- **Golden Cross + RSI < 30** → "Kaufen"
- **Death Cross + RSI > 70** → "Verkaufen"

Shiny-App-Screenshot (Platzhalter für deine eigene App!)

Weiter Vorschläge für Tradingfreaks

- **Backtesting:** Füge eine Performance-Simulation hinzu (z. B. Vergleich mit Buy-and-Hold).
- **Mehr ETFs:** Erweitere die Auswahl um internationale Indizes (z. B. `EUFN` für Europa).